

Memory-Efficient Torch Inference for AlphaGenome

July 3, 2026

Abstract

We evaluated a sequence of memory-saving inference strategies for a Torch implementation of AlphaGenome using a recreated merged human brain development ATAC checkpoint. The strategies combine bfloat16 execution, precomputed standardized convolutions, Triton int8 weight-only one-dimensional convolutions, an encoder path that omits unneeded intermediate tensors, a custom no-indices Triton max-pooling kernel, and fused high-resolution encoder blocks. On 131 kb input windows at batch size 32, the most memory-efficient successful configuration reduced observed peak device memory from 92.6 GiB to 25.2 GiB relative to default Torch inference, while preserving differential Pearson to within 0.001. On 1 Mb input windows at batch size 2, the default path exceeded 48 GiB, whereas the memory-efficient configurations were between 24.1 GiB and 33.4 GiB observed peak memory.

1 Experimental setup

All experiments used the recreated merged Torch checkpoint from the best LoRA+LoCon finetuning run. We evaluated the human brain development ATAC head at resolution 128 on full valid and test splits. For 131 kb input windows, batch size was 32 and the evaluation contained 2162 examples. For 1 Mb input windows, a valid/test-only target cache was built, batch size was 2, and the evaluation contained 269 examples. Peak device memory was sampled with `nvidia-smi`; PyTorch allocated and reserved memory were recorded from CUDA allocator statistics.

2 Memory-saving strategies

The evaluated strategies are cumulative unless otherwise indicated.

1. **Bfloat16 parameters and activations.** The `bf16_params` baseline applies the aggressive bfloat16 Torch dtype policy for inference.
2. **Precomputed standardized convolutions and Triton int8 weight-only Conv1d.** Standardized convolution weights are materialized once for inference and replaced by direct effective convolutions. Eligible wide convolutions are then replaced by Triton int8 weight-only Conv1d kernels with floating-point output activations.
3. **No-intermediates encoder path.** When only 128 bp predictions are requested, U-Net skip tensors that are not consumed are not retained.
4. **Triton no-indices max-pooling.** The encoder max-pooling layer is replaced by a Triton implementation for kernel size 2 and stride 2 that does not materialize PyTorch max-pool indices.

5. **Fused DNA embedder block.** The full-resolution `encoder.dna_embedder.block` is fused into one Triton operation implementing RMSBatchNorm, the JAX GELU approximation, and int8 weight-only Conv1d.
6. **Fused first downsampling block.** The `encoder.down_blocks.0` module is replaced by a wrapper using fused ConvBlocks and in-place residual accumulation in inference.

3 Results

3.1 131 kb input windows

Table 1: Full valid/test evaluation for 131 kb windows at batch size 32.

Strategy	Ex./s	Peak MiB	Alloc MiB	Reserved MiB	Valid r	Test r
Default	9.41	92631	63140	91928	0.79893	0.80891
Bfloat16	12.02	66935	42415	66248	0.79839	0.80874
Triton Conv1d	8.49	43611	27879	42924	0.79842	0.80877
No intermediates	8.51	43611	25319	42924	0.79842	0.80877
Triton pool	8.78	34907	25319	34220	0.79842	0.80877
Fused embedder	8.03	32347	21735	31660	0.79845	0.80872
Fused down0	8.11	31323	25319	30636	0.79829	0.80860
Fused embedder+down0	7.45	25179	19174	24492	0.79834	0.80860

At 131 kb, the most memory-efficient configuration reduced observed peak memory from 92.6 GiB to 25.2 GiB relative to default inference and from 66.9 GiB to 25.2 GiB relative to bfloat16 inference. The best throughput among configurations below 48 GiB was the Triton-pool strategy, at 8.78 examples/s and 34.9 GiB.

3.2 1 Mb input windows

Table 2: Full valid/test evaluation for 1 Mb windows at batch size 2.

Strategy	Ex./s	Peak MiB	Alloc MiB	Reserved MiB	Valid r	Test r
Default	0.721	52595	38507	51908	0.81825	0.82707
Bfloat16	0.937	47597	25445	46910	0.81522	0.82640
Triton Conv1d	0.736	33359	22508	32672	0.81558	0.82662
No intermediates	0.749	29263	15452	28576	0.81558	0.82662
Triton pool	0.753	30799	15452	30112	0.81558	0.82662
Fused embedder	0.701	29519	15453	28832	0.81554	0.82658
Fused down0	0.697	27215	15452	26528	0.81541	0.82643
Fused embedder+down0	0.650	24143	15452	23456	0.81534	0.82634

At 1 Mb, default inference exceeded 48 GiB observed peak memory. Bfloat16 inference fit just under 48 GiB and was the fastest measured option. The Triton/no-intermediates configurations reduced live allocated memory further, to approximately 15.5 GiB, while remaining well below 48 GiB observed memory. The fused kernels primarily reduced allocator-reserved and observed memory at this longer context length, but also reduced throughput.

4 Discussion

The most robust practical configuration is the combination of bfloat16, precomputed standardized convolutions, Triton int8 weight-only Conv1d, the 128 bp no-intermediates encoder path, and Triton no-indices max-pooling. This configuration preserved differential Pearson relative to the bfloat16/Triton Conv1d baseline and kept peak memory below 48 GiB for both tested input lengths. The fused high-resolution blocks are useful when minimizing observed memory is more important than maximizing throughput. For 1 Mb inputs, the tower and attention computations dominate a larger share of runtime, so convolution fusions provide less throughput benefit than at shorter context length.